

Sql commands

```
CREATE DATABASE Transport_System;
USE Transport_System;
```

```
CREATE TABLE User (
  id INT AUTO_INCREMENT PRIMARY KEY,
  username VARCHAR(50) UNIQUE NOT NULL,
  password VARCHAR(255) NOT NULL,
  full_name VARCHAR(100),
  role VARCHAR(20),
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP
);
```

```
CREATE TABLE Truck (
  id INT AUTO_INCREMENT PRIMARY KEY,
  number_plate VARCHAR(20) UNIQUE NOT NULL,
  type VARCHAR(50),
  model VARCHAR(50),
  capacity_kg INT,
  status ENUM('available','in_trip','maintenance') DEFAULT 'available',
  purchase_date DATE
);
```

```
CREATE TABLE Driver (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  phone VARCHAR(20),
  license_no VARCHAR(50),
  hire_date DATE,
  status ENUM('active','inactive') DEFAULT 'active',
  commission_type ENUM('fixed','percentage'),
  commission_value DECIMAL(10,2)
);
```

```
CREATE TABLE DriverTruckHistory (
  id INT AUTO_INCREMENT PRIMARY KEY,
  driver_id INT,
  truck_id INT,
  start_date DATE,
  end_date DATE,
  FOREIGN KEY (driver_id) REFERENCES Driver(id),
  FOREIGN KEY (truck_id) REFERENCES Truck(id)
);
```

```
CREATE TABLE Broker (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(100),
  phone VARCHAR(20),
  company_name VARCHAR(100),
  address TEXT
);
```

```
CREATE TABLE Company (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  contact_person VARCHAR(100),  
  phone VARCHAR(20),  
  address TEXT  
);
```

```
CREATE TABLE Customer (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  name VARCHAR(100),  
  phone VARCHAR(20),  
  address TEXT  
);
```

```
CREATE TABLE Trip (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  truck_id INT NOT NULL,  
  start_date DATE,  
  end_date DATE,  
  start_location VARCHAR(255),  
  end_location VARCHAR(255),  
  status ENUM('planned','ongoing','completed') DEFAULT 'planned',  
  notes TEXT,  
  created_by INT,  
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,  
  FOREIGN KEY (truck_id) REFERENCES Truck(id),  
  FOREIGN KEY (created_by) REFERENCES User(id)  
);
```

```
CREATE TABLE TripDriver (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  trip_id INT,  
  driver_id INT,  
  role ENUM('main_driver','co_driver'),  
  FOREIGN KEY (trip_id) REFERENCES Trip(id) ON DELETE CASCADE,  
  FOREIGN KEY (driver_id) REFERENCES Driver(id)  
);
```

```
CREATE TABLE Shipment (  
  id INT AUTO_INCREMENT PRIMARY KEY,  
  trip_id INT,  
  broker_id INT,  
  company_id INT,  
  customer_id INT,  
  pickup_location VARCHAR(255),  
  dropoff_location VARCHAR(255),  
  goods_type VARCHAR(100),  
  weight_kg DECIMAL(10,2),  
  rate_per_kg DECIMAL(10,2),
```

```
total_amount DECIMAL(12,2),
pickup_date DATE,
delivery_date DATE,
status ENUM('pending','picked','delivered') DEFAULT 'pending',
created_by INT,
created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
FOREIGN KEY (trip_id) REFERENCES Trip(id) ON DELETE CASCADE,
FOREIGN KEY (broker_id) REFERENCES Broker(id),
FOREIGN KEY (company_id) REFERENCES Company(id),
FOREIGN KEY (customer_id) REFERENCES Customer(id),
FOREIGN KEY (created_by) REFERENCES User(id)
);
```

```
CREATE TABLE TripStop (
  id INT AUTO_INCREMENT PRIMARY KEY,
  trip_id INT,
  location VARCHAR(255),
  stop_type ENUM('pickup','drop','rest'),
  sequence_number INT,
  arrival_time DATETIME,
  departure_time DATETIME,
  FOREIGN KEY (trip_id) REFERENCES Trip(id) ON DELETE CASCADE
);
```

```
CREATE TABLE Expense (
  id INT AUTO_INCREMENT PRIMARY KEY,
  trip_id INT,
  type ENUM('fuel','toll','food','stay','repair','other'),
  amount DECIMAL(10,2),
  date DATE,
  description TEXT,
  receipt_image VARCHAR(255),
  created_by INT,
  created_at DATETIME DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (trip_id) REFERENCES Trip(id) ON DELETE CASCADE,
  FOREIGN KEY (created_by) REFERENCES User(id)
);
```

```
CREATE TABLE Invoice (
  id INT AUTO_INCREMENT PRIMARY KEY,
  trip_id INT,
  total_amount DECIMAL(12,2),
  issue_date DATE,
  status ENUM('pending','paid','partial') DEFAULT 'pending',
  FOREIGN KEY (trip_id) REFERENCES Trip(id)
);
```

```
CREATE TABLE InvoiceItem (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT,
  shipment_id INT,
```

```
amount DECIMAL(12,2),
FOREIGN KEY (invoice_id) REFERENCES Invoice(id) ON DELETE CASCADE,
FOREIGN KEY (shipment_id) REFERENCES Shipment(id)
);
```

```
CREATE TABLE Payment (
  id INT AUTO_INCREMENT PRIMARY KEY,
  invoice_id INT,
  amount DECIMAL(12,2),
  payment_date DATE,
  method ENUM('cash','bank','online'),
  FOREIGN KEY (invoice_id) REFERENCES Invoice(id) ON DELETE CASCADE
);
```

```
CREATE TABLE DriverPayout (
  id INT AUTO_INCREMENT PRIMARY KEY,
  driver_id INT,
  trip_id INT,
  role ENUM('main_driver','co_driver'),
  amount DECIMAL(12,2),
  status ENUM('pending','paid') DEFAULT 'pending',
  payment_date DATE,
  FOREIGN KEY (driver_id) REFERENCES Driver(id),
  FOREIGN KEY (trip_id) REFERENCES Trip(id)
);
```

```
CREATE TABLE TripSummary (
  id INT AUTO_INCREMENT PRIMARY KEY,
  trip_id INT UNIQUE,
  total_income DECIMAL(12,2),
  total_expense DECIMAL(12,2),
  profit DECIMAL(12,2),
  FOREIGN KEY (trip_id) REFERENCES Trip(id) ON DELETE CASCADE
);
```

```
INSERT INTO User (id, username, password, full_name, role, created_at) VALUES
(1, 'admin01', 'pass123', 'Ali Khan', 'admin', '2026-01-10'),
(2, 'dispatcher1', 'pass456', 'Sara Ahmed', 'dispatcher', '2026-02-05');
```

```
INSERT INTO Truck (id, number_plate, type, model, capacity_kg, status, purchase_date) VALUES
(1, 'ABC-123', 'Flatbed', 'Hino 300', 10000, 'available', '2022-05-15'),
(2, 'XYZ-789', 'Container', 'Isuzu NPR', 8000, 'maintenance', '2023-03-20');
```

```
INSERT INTO Driver (id, name, phone, license_no, hire_date, status, commission_type, commission_value)
VALUES
(1, 'Imran Ali', '03001234567', 'LIC12345', '2024-01-01', 'active', 'percentage', 10),
(2, 'Bilal Hussain', '03007654321', 'LIC67890', '2025-06-12', 'active', 'fixed', 5000);
```

```
INSERT INTO DriverTruckHistory (id, driver_id, truck_id, start_date, end_date) VALUES
(1, 1, 1, '2025-01-01', '2025-12-31'),
(2, 2, 2, '2025-07-01', NULL);
```

```
INSERT INTO Broker (id, name, phone, company_name, address) VALUES
(1, 'Kamran Shah', '03005556677', 'Fast Logistics', 'Karachi'),
(2, 'Nadia Malik', '03009998877', 'CargoPro', 'Lahore');
```

```
INSERT INTO Company (id, name, contact_person, phone, address) VALUES
(1, 'Textile Mills Ltd', 'Zeeshan', '03001112233', 'Faisalabad'),
(2, 'Agro Foods', 'Farah', '03002223344', 'Multan');
```

```
INSERT INTO Customer (id, name, phone, address) VALUES
(1, 'Ahmed Traders', '03003334455', 'Karachi'),
(2, 'Global Mart', '03004445566', 'Islamabad');
```

```
INSERT INTO Trip (id, truck_id, start_date, end_date, start_location, end_location, status, notes,
created_by) VALUES
(1, 1, '2026-03-01', '2026-03-05', 'Karachi', 'Lahore', 'completed', 'Smooth trip', 2),
(2, 2, '2026-03-10', '2026-03-12', 'Lahore', 'Islamabad', 'ongoing', 'Minor delay', 2);
```

```
INSERT INTO TripDriver (id, trip_id, driver_id, role) VALUES
(1, 1, 1, 'main_driver'),
(2, 1, 2, 'co_driver'),
(3, 2, 2, 'main_driver');
```

```
INSERT INTO Shipment (id, trip_id, broker_id, company_id, customer_id, pickup_location,
dropoff_location, goods_type, weight_kg, rate_per_kg, total_amount, pickup_date, delivery_date, status,
created_by) VALUES
(1, 1, 1, 1, 1, 'Karachi', 'Lahore', 'Textiles', 5000, 20, 100000, '2026-03-01', '2026-03-05', 'delivered', 2),
(2, 2, 2, 2, 2, 'Lahore', 'Islamabad', 'Food Products', 3000, 15, 45000, '2026-03-10', NULL, 'picked', 2);
```

```
INSERT INTO TripStop (id, trip_id, location, stop_type, sequence_number, arrival_time, departure_time)
VALUES
(1, 1, 'Hyderabad', 'rest', 1, '2026-03-02 10:00', '2026-03-02 11:00'),
(2, 1, 'Multan', 'rest', 2, '2026-03-03 18:00', '2026-03-04 08:00');
```

```
INSERT INTO Expense (id, trip_id, type, amount, date, description, receipt_image, created_by) VALUES
(1, 1, 'fuel', 20000, '2026-03-02', 'Diesel refill', NULL, 2),
(2, 1, 'toll', 5000, '2026-03-03', 'Highway tolls', NULL, 2);
```

```
INSERT INTO Invoice (id, trip_id, total_amount, issue_date, status) VALUES
(1, 1, 100000, '2026-03-06', 'paid'),
(2, 2, 45000, '2026-03-13', 'pending');
```

```
INSERT INTO InvoiceItem (id, invoice_id, shipment_id, amount) VALUES
(1, 1, 1, 100000),
(2, 2, 2, 45000);
```

```
INSERT INTO Payment (id, invoice_id, amount, payment_date, method) VALUES
(1, 1, 100000, '2026-03-07', 'bank');
```

```
INSERT INTO DriverPayout (id, driver_id, trip_id, role, amount, status, payment_date) VALUES
(1, 1, 1, 'main_driver', 10000, 'paid', '2026-03-08');
```

```
(2, 2, 1, 'co_driver', 5000, 'paid', '2026-03-08');
```

```
INSERT INTO TripSummary (id, trip_id, total_income, total_expense, profit) VALUES  
(1, 1, 100000, 25000, 75000),  
(2, 2, 45000, 0, 45000);
```

```
-- PROFIT TREND WITH MOVING AVERAGE
```

```
WITH TripProfit AS (  
    SELECT  
        t.id,  
        t.start_date,  
        SUM(s.total_amount) - IFNULL(SUM(e.amount),0) AS profit  
    FROM Trip t  
    JOIN Shipment s ON t.id = s.trip_id  
    LEFT JOIN Expense e ON t.id = e.trip_id  
    GROUP BY t.id, t.start_date  
)  
SELECT  
    id,  
    start_date,  
    profit,  
    AVG(profit) OVER (  
        ORDER BY start_date  
        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW  
    ) AS moving_avg_profit  
FROM TripProfit;
```

```
-- driver activity report
```

```
SELECT  
    d.id,  
    d.name,  
    trip_stats.trip_count,  
    CASE  
        WHEN trip_stats.trip_count >= 10 THEN 'High Activity'  
        WHEN trip_stats.trip_count BETWEEN 5 AND 9 THEN 'Moderate Activity'  
        ELSE 'Low Activity'  
    END AS activity_level  
FROM Driver d  
JOIN (  
    SELECT  
        td.driver_id,  
        COUNT(td.trip_id) AS trip_count  
    FROM TripDriver td  
    GROUP BY td.driver_id  
) AS trip_stats  
ON d.id = trip_stats.driver_id;
```

```
-- FIND TRIPS WHERE EXPENSE SPIKE OCCURRED
```

```
SELECT  
    e.trip_id,  
    e.date,
```

```

    e.amount
FROM Expense e
WHERE e.amount > (
    SELECT AVG(e2.amount)
    FROM Expense e2
    WHERE e2.trip_id = e.trip_id
)
AND e.date = (
    SELECT MAX(e3.date)
    FROM Expense e3
    WHERE e3.trip_id = e.trip_id
);

```

```

-- Create a view that restricts access to only trip profits
CREATE VIEW TripProfitView AS
SELECT
    t.id AS trip_id,
    t.start_date,
    SUM(s.total_amount) - IFNULL(SUM(e.amount),0) AS profit
FROM Trip t
JOIN Shipment s ON t.id = s.trip_id
LEFT JOIN Expense e ON t.id = e.trip_id
GROUP BY t.id, t.start_date;
SELECT
    trip_id,
    start_date,
    profit
FROM TripProfitView
WHERE profit < 0;

```

```

-- Create a stored procedure to get trip profit by trip_id
DELIMITER $$
CREATE PROCEDURE GetTripProfit(IN tripId INT)
BEGIN
    SELECT
        t.id AS trip_id,
        t.start_date,
        SUM(s.total_amount) - IFNULL(SUM(e.amount),0) AS profit
    FROM Trip t
    JOIN Shipment s ON t.id = s.trip_id
    LEFT JOIN Expense e ON t.id = e.trip_id
    WHERE t.id = tripId
    GROUP BY t.id, t.start_date;
END $$
DELIMITER ;

```

```

CALL GetTripProfit(101);  -- Call the procedure

```

```

-- Create composite index to optimize joins and filtering
CREATE INDEX idx_expense_trip_date ON Expense(trip_id, date);

```



```

SELECT
  t.id AS trip_id,
  t.start_date,
  SUM(e.amount) AS total_expense,
  CASE
    WHEN SUM(e.amount) > (
      SELECT AVG(total_expense)
      FROM (
        SELECT SUM(e2.amount) AS total_expense
        FROM Expense e2
        GROUP BY e2.trip_id
      ) AS sub_avg
    ) THEN 'Above Average'
    ELSE 'Below Average'
  END AS expense_category,
  RANK() OVER (ORDER BY SUM(e.amount) DESC) AS expense_rank
FROM Trip t
JOIN Expense e ON t.id = e.trip_id
WHERE e.date = (
  SELECT MAX(e3.date)
  FROM Expense e3
  WHERE e3.trip_id = t.id
)
GROUP BY t.id, t.start_date;

```

-- Driver Performance Categorization via Stored Procedure

DELIMITER \$\$

CREATE PROCEDURE DriverPerformanceReport()

BEGIN

SELECT

d.id,

d.name,

COUNT(td.trip_id) AS trip_count,

CASE

WHEN COUNT(td.trip_id) >= 15 THEN 'Top Performer'

WHEN COUNT(td.trip_id) BETWEEN 5 AND 14 THEN 'Average Performer'

ELSE 'Low Performer'

END AS performance_category

FROM Driver d

LEFT JOIN TripDriver td ON d.id = td.driver_id

GROUP BY d.id;

END \$\$

DELIMITER ;

CALL DriverPerformanceReport();

-- Create a view showing only high-value shipments

CREATE VIEW HighValueShipments AS

SELECT

s.id,

s.trip_id,

s.goods_type,

```

    s.total_amount
FROM Shipment s
WHERE s.total_amount > (
    SELECT AVG(s2.total_amount)
    FROM Shipment s2
);
SELECT * FROM HighValueShipments;

```

```

-- Create index to optimize shipment lookups
CREATE INDEX idx_shipment_trip ON Shipment(trip_id);
SELECT
    s.trip_id,
    s.goods_type,
    s.total_amount,
    RANK() OVER (PARTITION BY s.trip_id ORDER BY s.total_amount DESC) AS shipment_rank
FROM Shipment s;

```

```

use Transport_System;
-- rip Profit Calculation Using Correlated Subqueries
SELECT
    t.id AS trip_id,
    t.start_date,
    (SELECT SUM(e.amount)
     FROM Expense e
     WHERE e.trip_id = t.id) AS total_expense,
    (SELECT SUM(s.total_amount)
     FROM Shipment s
     WHERE s.trip_id = t.id) AS total_income,
    (SELECT SUM(s.total_amount)
     FROM Shipment s
     WHERE s.trip_id = t.id) -
    (SELECT SUM(e.amount)
     FROM Expense e
     WHERE e.trip_id = t.id) AS profit
FROM Trip t
WHERE t.start_date > '2026-01-01';

```

```

SELECT
    t.id AS trip_id,
    t.start_date,
    SUM(s.total_amount) - SUM(e.amount) AS profit,
    CASE
        WHEN SUM(s.total_amount) > SUM(e.amount) THEN 'Profitable'
        WHEN SUM(s.total_amount) < SUM(e.amount) THEN 'Loss'
        ELSE 'Break-Even'
    END AS trip_status
FROM
    Trip t

```

```
JOIN Shipment s ON t.id = s.trip_id
JOIN Expense e ON t.id = e.trip_id
WHERE
  t.start_date > '2026-01-01'
GROUP BY
  t.id, t.start_date
ORDER BY
  profit DESC;
```